

Weekly Progress Report

Jac LLM

September 16, 2026



Agenda

1. Dataset



- Data generation pipeline
- Format and task types.

2. Model Output



- Latest output
- Current limitation

3. Statistics



- Metrics and evaluation.
- Insight from pilot experiment

4. Discussion



- Bottleneck
- Next steps

Dataset

2 Axes of Model Capability

Lexical /
Syntax

Semantic

Idiomatic

System-
Level



Inference Capability

Basic
Functions

Graph &
Iteration

OSP

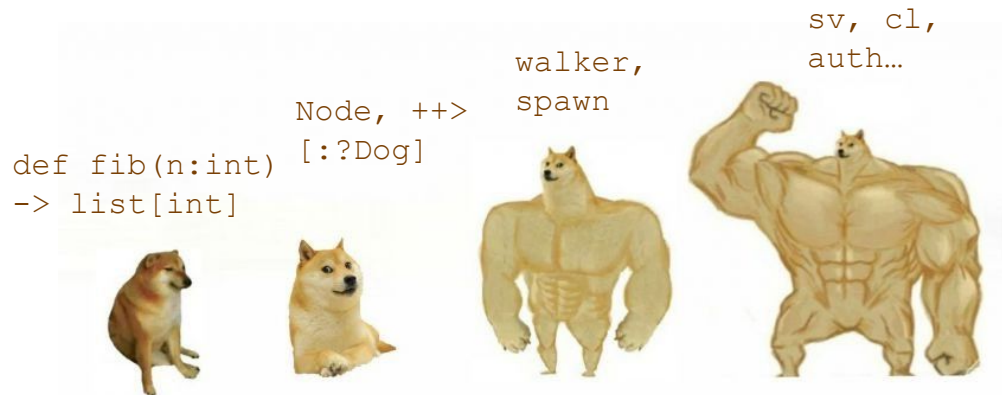
Fullstack



Knowledge Categories

Dataset

2 Axes of Model Capability



Knowledge Categories

Grammar
punctuation

Knows the
meaning behind
the symbol

Good practices
of writing jac

User intent →
working e2e app

Inference Capability

1. Dataset

- a. Format and task types.
- b. Data generation pipeline.

Dataset

What we currently have

Fixed jac version to 0.36.X

CPT

- Jac official docs
- Nitin's 10k jac functions
- **JacHack / App**

SFT

- Code completion
- Py 2 Jac
- JS 2 Jac
- Code gen (Opus)
- QA (Opus)
- **OSP**

TODO

- Git diff
- Claude Session
- CoT style debugging
- Runtime

Dataset

CPT

Classified with Regex,
comment and docstrings excluded

Source	#train	Avg. #tokens	Function	Graph	OSP	Fullstack
Jac docs	646	4k	-	-	-	-
Nitin's 10k jac func.	6k	100	6k	-	-	-
Jac Hack	???	???				all

Total	3M tokens					
-------	-----------	--	--	--	--	--

SFT Task Type

Task type	Instruction Example
code_completion	Complete the following function in jac <code>"""<Docstring>""" def cpu_statistics(last: list[list[int]], ref: list[list[int]]) -> list[float] {</code>
py2jac	Convert the following python code to jac <code>``python <code>``</code>
js2jac	Convert the following js code to jac <code>``js <code>``</code>
code_gen	Write a Person object with a name, an age, and an optional email. Give it a method that returns a friendly greeting....
qa	Can edges carry data and methods? What does the `Weighted` edge define?

Dataset

SFT

Classified with Regex,
comment and docstrings excluded

Task type	#train	Avg. #tokens	Function	Graph	OSP	Fullstack
code_completion	~6k	200	6k	-	-	-
py2jac	~388	200	328	-	60	-
js2jac	~3.6k	300	3k	-	-	600
code_gen	~18k	500	600	400	500	300
qa	~4k	200	N/A	N/A	N/A	N/A
Sum	32k		10k	400	560	1k

Total	12M tokens
--------------	-------------------

New Dataset

OSP

- What

- E
- M
- C

- Exam

- M

- Data

- M

2. Model Output

- a. Latest output
- b. Current limitation

Mixture of Expert: 30B-A3B

Qwen3 Coder

Spectrum - Select 16 layers with SNR probing

Target - QKVO proj, up/down/gate proj, router

LoRA - rank = 16

Epoch -

CPT >> SFT



Model Output

Basic Functions

```
jac
def fib(n: int) -> list[int] {
  fib2 = 0;
  fib1 = 1;
  print(fib1);
  print(fib2);
  for i in range(2, n) {
    fib2 = fib1;
    fib1 = fib2 + fib1;
    print(fib1);
  }
  return fib;
}
```

📄 ✎ ↺ 🗑️ 🔊 ... 29.9 tok/s

Prompt

write fibonacci in jac. def fib(n:int) -> list[int] {

Problem

Logic looks correct, but it did not use list properly and return the incorrect things..

Model Output

Front-end

```
node Person {
  has name: str = "";
  has bio: str = "";
  has showBio: bool = False;

  def toggle_bio() -> None {
    self.showBio = not self.showBio;
  }
}

def:pub PersonCard() -> JsxElement {
  p = Person(name="Jaseci", bio="Full-stack developer with 5+ years of experience");
  return <div className="person-card">
    <h1>{p.name}</h1>
    <p>{p.bio}</p>
    <button onClick={p.toggle_bio}>
      Toggle bio
    </button>
    <div className="bio">
      Passionate about building things that matter.
    </div>
  </div>
  ↓
}
```

Prompt

can you write a personal webpage for me as a jsx component in jac?

Model Output

Graph & OSP

Prompt

write in python a dfs search over a tree

Problem

Filter is required for for-loop.

```
node Item {
    has value: int = 0;
}

walker Searcher {
    has start: int = 0;
    has visited: list = [];
    has level: int = 0;

    can explore with Root entry {
        visit [-->];
    }

    can search with Item entry {
        # Queue all outgoing nodes before descending
        queue = [here.value];
        for child in [here -->] {
            queue.append(child.value);
        }
        if len(queue) == 0 {
            print(f"Reached a leaf: {here.value}");
        } else {
            # Process this node, then recurse into children
            self.level += 1;
            search(queue);
        }
    }
}

with entry {
    # Build a small tree
    root ++> Item(value=1) ++> Item(value=2) ++> Item(value=3);
    root ++> Item(value=4);
    root ++> Item(value=5);

    # Start search from root
    root spawn Searcher();
}
```

Model Output

Graph & OSP

Prompt

write in python a dfs search over a tree

Problem

Filter is required for for-loop.

```
node Item {
    has value: int = 0;
}

walker Searcher {
    has start: int = 0;
    has visited: list = [];
    has level: int = 0;

    can explore with Root entry {
        visit [-->];
    }

    can search with Item entry {
        # Queue all outgoing nodes before descending
        queue = [here.value];
        for child in [here -->] {
            queue.append(child.value);
        }
        if len(queue) == 0 {
            print(f"Reached a leaf: {here.value}");
        } else {
            # Process this node, then recurse into children
            self.level += 1;
            search(queue);
        }
    }
}

with entry {
    # Build a small tree
    root ++> Item(value=1) ++> Item(value=2) ++> Item(value=3);
    root ++> Item(value=4);
    root ++> Item(value=5);

    # Start search from root
    root spawn Searcher();
}
```

Dense Model: 9B

Ornith 1.5 (Qwen 3.5 based)

Blocks - All 36 Blocks

Target - QKVO proj, up/down/gate proj

RSLoRA - rank = 64

Epoch - 1

CPT >> SFT



Model Output

Model

- **Ornith-1.5-9B-4bit (bnb)**

- Based on Qwen 3.5 9B
- This is just a pilot / probing experiment to find out what is missing and potential blockers, and the data is relatively fragmented

- **Training**

- 1 epoch for each dataset
- Max sequence length = 4096 (~16k char, 3k words)
(Typical set up: CPT 16k, SFT 8k, long context: 32k trajectories)
- **CPT**: jac official docs chunks, Nitin's 10k function level data
- **SFT**: code completion, py2jac, js2jac, code gen, QA
- **RL**: coming soon...

Model Output

Basic Functions

```
jac
def fib(n: int) -> list[int] {
  fib2 = 0;
  fib1 = 1;
  print(fib1);
  print(fib2);
  for i in range(2, n) {
    fib2 = fib1;
    fib1 = fib2 + fib1;
    print(fib1);
  }
  return fib;
}
```

📄 ✎ ↺ 🗑️ 🔊 ... 29.9 tok/s

Prompt

write fibonacci in jac. def fib(n:int) -> list[int] {

Problem

Logic looks correct, but it did not use list properly and return the incorrect things..

Model Output

Front-end

```
node Person {
  has name: str = "";
  has bio: str = "";
  has showBio: bool = False;

  def toggle_bio() -> None {
    self.showBio = not self.showBio;
  }
}

def:pub PersonCard() -> JsxElement {
  p = Person(name="Jaseci", bio="Full-stack developer with 5+ years of experience");
  return <div className="person-card">
    <h1>{p.name}</h1>
    <p>{p.bio}</p>
    <button onClick={p.toggle_bio}>
      Toggle bio
    </button>
    <div className="bio">
      Passionate about building things that matter.
    </div>
  </div>
  ↓
}
```

Prompt

can you write a personal webpage for me as a jsx component in jac?

Model Output

Graph & OSP

Prompt

write in python a dfs search over a tree

Problem

Filter is required for for-loop.

```
node Item {
    has value: int = 0;
}

walker Searcher {
    has start: int = 0;
    has visited: list = [];
    has level: int = 0;

    can explore with Root entry {
        visit [-->];
    }

    can search with Item entry {
        # Queue all outgoing nodes before descending
        queue = [here.value];
        for child in [here -->] {
            queue.append(child.value);
        }
        if len(queue) == 0 {
            print(f"Reached a leaf: {here.value}");
        } else {
            # Process this node, then recurse into children
            self.level += 1;
            search(queue);
        }
    }
}

with entry {
    # Build a small tree
    root ++> Item(value=1) ++> Item(value=2) ++> Item(value=3);
    root ++> Item(value=4);
    root ++> Item(value=5);

    # Start search from root
    root spawn Searcher();
}
```

Model Output

Where we are?

- **Model learns the syntax for all categories**
(function, graph, OSP, fullstacks)
- **Limited semantic understanding**
→ CoT dataset with runtime behavior
- **Limited debugging / self correction capabilities**
→ Learn from compiler feedback
→ Git diff dataset, or teacher model
- **Ignore details of instructions**
→ More SFT, Claude Sessions...



Lexical /
Syntax

Semantic

Idiomatic

System-
Level



Model Capability

3. Statistics

- a. Metrics and evaluation
- b. Insight from pilot experiment

Metrics

- **Training**

- a. Loss, Validation Loss: Does the model fit/overfit the data
- b. Grad Norm: Stability and Robustness of training

- **jac check**

pass@1, pass@k, can discuss more on fine-grained error classes

- **jac test**

% test case passed, % AC

- **jac start**

For full stack apps, check HTTP 200, white test, mock the back, e2e.

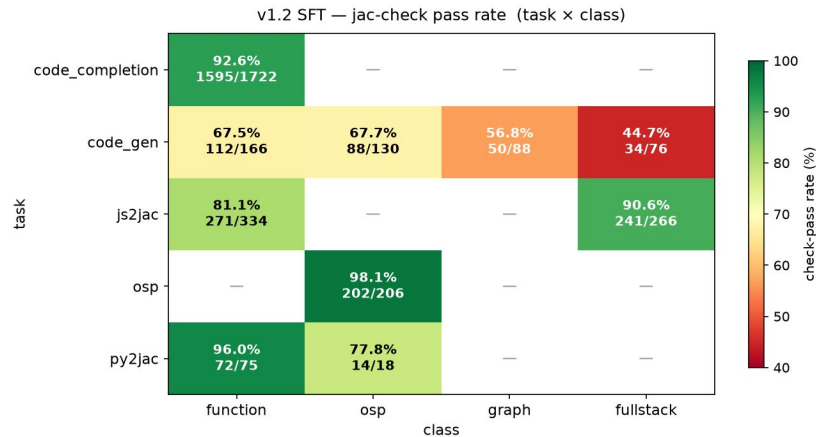
Statistics - Qwen3 30B A3B

Evaluation - jac check

On validation dataset

has_block means we find ```jac <code>``` block

cl {} existing in dataset for fullstack data

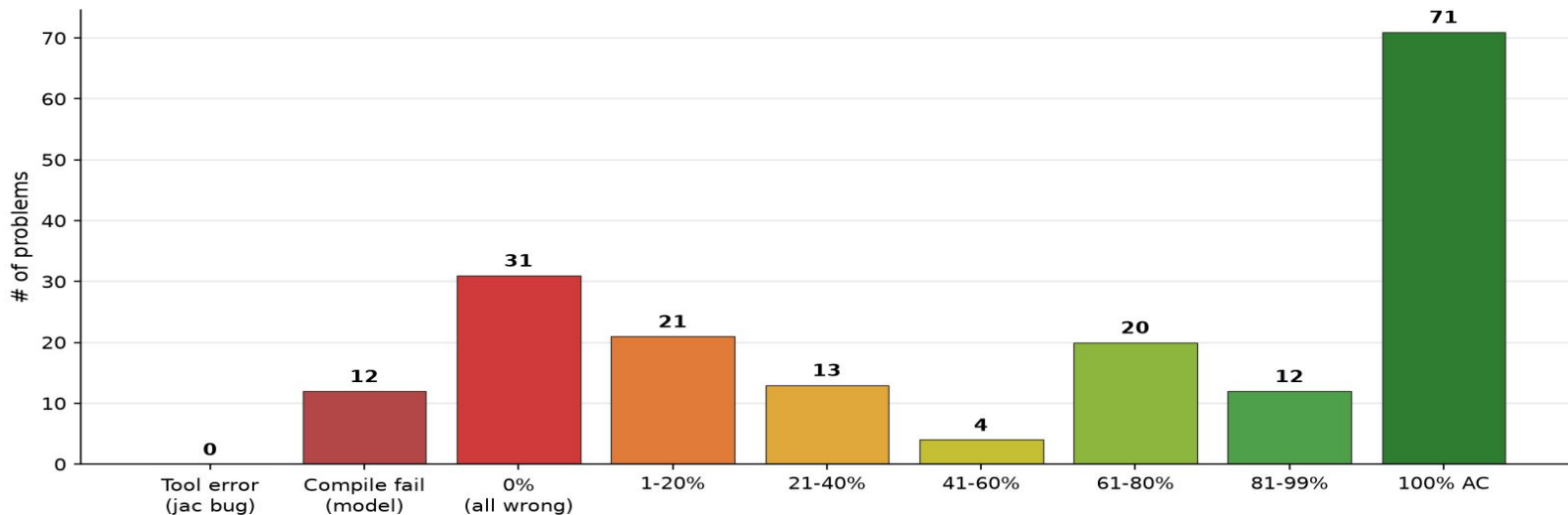


Task type	Function		Graph		OSP		Fullstack	
code_completion	1348/1523	89%	-	-	-	-	-	-
py2jac	271/328	83%	-	-	25/60	41%	-	-
js2jac	398/754	53%	-	-	-	-	46/168	27%
code_gen	105/166	63%	50/88	57%	67/130	52%	29/76	38%

Statistics - Qwen3 30B A3B

Evaluation - jac test

Testcase pass-rate distribution
0910-trueseq-r64 SFT on function-eval v1 dev, jac 0.36.0



AC = 71/184 (38.6%) | partial credit = 70 problems | testcase pass rate = 64.6%

Statistics - Ornith1.5 9B Dense

Evaluation - jac check

On validation dataset

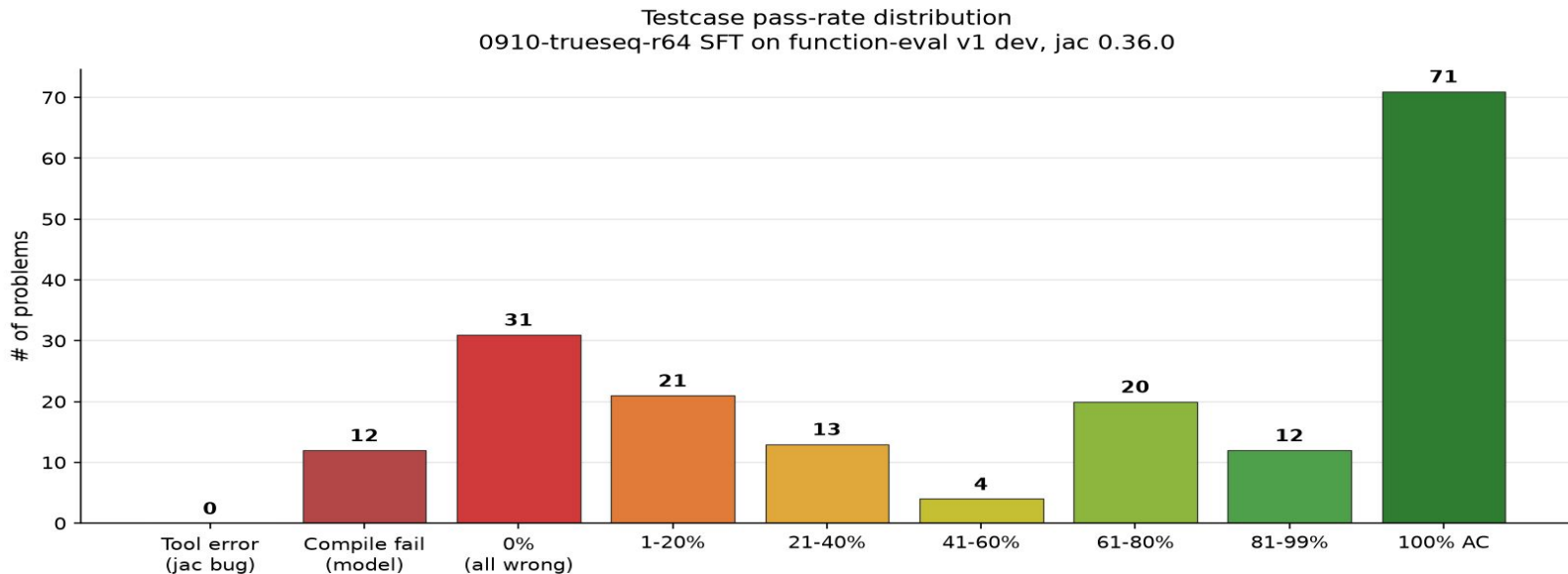
has_block means we find `jac <code>` block

cl {} existing in dataset for fullstack data

Task type	Function		Graph		OSP		Fullstack	
code_completion	1348/1523	89%	-	-	-	-	-	-
py2jac	271/328	83%	-	-	25/60	41%	-	-
js2jac	398/754	53%	-	-	-	-	46/168	27%
code_gen	105/166	63%	50/88	57%	67/130	52%	29/76	38%

Statistics - Ornith1.5 9B Dense

Evaluation - jac test



AC = 71/184 (38.6%) | partial credit = 70 problems | testcase pass rate = 64.6%

4. Discussion

- a. Bottleneck
- b. Next steps

Discussion

Bottleneck

- **Model Inference Capability**

- Ornith is stubborn and not following detailed or specific instructions.
- Dataset for semantic understanding (maybe CoT or Runtime)
- Idiomatic Jac

- **Data Generation & Validation**

- Scalable jac app data generation
- Generate test cases
- Check the app is working properly

- **Computation Resources**

- For both training and data gen

Discussion

Next steps

- **Data Gen and Preprocessing**
 - Collecting new ver of docs and jac repo
 - Mining from Claude Session and git diff
 - Jac Project Factory
- **Reinforcement Learning**
 - DPO for offline training
 - GRPO for online training (Jac Playground)
 - Learning from runtime and compiler feedback
- **Self-evolving Jac Agent**

Thank You !